

SOLID Design Principles

SOLID is a set of 5 design principles for writing clean, maintainable, and flexible object-oriented code. The name is an acronym created by Robert C. Martin (Uncle Bob).

Following these principles makes your Java code easier to understand, test, and change over time.

Overview

Letter	Principle	Short meaning
S	Single Responsibility	A class should have only one reason to change
O	Open/Closed	Open for extension, closed for modification
L	Liskov Substitution	Subtypes must be substitutable for their base types
I	Interface Segregation	Many small interfaces are better than one big one
D	Dependency Inversion	Depend on abstractions, not on concrete classes

1. Single Responsibility Principle (SRP)

Theory

A class should have only one reason to change. It should do only one job. If a class has many responsibilities, a change in one area can break the others.

Before (violates SRP)

One class does everything: calculate salary + print report + save to database.

```
public class Employee {  
    private String name;  
    private double salary;  
  
    public Employee(String name, double salary) {  
        this.name = name;  
        this.salary = salary;  
    }  
  
    // Responsibility 1: calculate salary
```

```
public double calculateSalary() {
    return salary * 1.1; // 10% bonus
}

// Responsibility 2: print report
public void printReport() {
    System.out.println("Employee: " + name + ", Salary: " + calculateSalary());
}

// Responsibility 3: save to database
public void saveToDatabase() {
    System.out.println("Saving " + name + " to database...");
}
}
```

After (follows SRP)

Each class has only one job.

```
public class Employee {
    private String name;
    private double salary;

    public Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }

    public String getName() { return name; }
    public double getSalary() { return salary; }
}

public class SalaryCalculator {
```

```

    public double calculate(Employee employee) {
        return employee.getSalary() * 1.1;
    }
}

public class EmployeeReport {
    public void print(Employee employee, double salary) {
        System.out.println("Employee: " + employee.getName()
            + ", Salary: " + salary);
    }
}

public class EmployeeRepository {
    public void save(Employee employee) {
        System.out.println("Saving " + employee.getName() + " to database...");
    }
}

```

2. Open/Closed Principle (OCP)

Theory

Software entities (classes, modules) should be open for extension but closed for modification. You should be able to add new behavior without changing existing code.

Before (violates OCP)

Every time we add a new shape we must change the AreaCalculator class.

```

public class AreaCalculator {
    public double calculateArea(Object shape) {
        if (shape instanceof Rectangle) {
            Rectangle r = (Rectangle) shape;
            return r.width * r.height;
        } else if (shape instanceof Circle) {
            Circle c = (Circle) shape;

```

```
        return Math.PI * c.radius * c.radius;
    }

    // If we add Triangle we must change this class again!
    return 0;
}
}
```

After (follows OCP)

We use an interface. New shapes only implement the interface — no change to existing code.

```
public interface Shape {
    double area();
}

public class Rectangle implements Shape {
    public double width;
    public double height;

    public Rectangle(double width, double height) {
        this.width = width;
        this.height = height;
    }

    @Override
    public double area() {
        return width * height;
    }
}

public class Circle implements Shape {
    public double radius;
```

```
public Circle(double radius) {
    this.radius = radius;
}

@Override
public double area() {
    return Math.PI * radius * radius;
}
}

// Now we can add Triangle without touching AreaCalculator
public class Triangle implements Shape {
    public double base;
    public double height;

    public Triangle(double base, double height) {
        this.base = base;
        this.height = height;
    }

    @Override
    public double area() {
        return 0.5 * base * height;
    }
}

public class AreaCalculator {
    public double calculateArea(Shape shape) {
        return shape.area(); // No modification needed
    }
}
```

3. Liskov Substitution Principle (LSP)

Theory

Objects of a superclass should be replaceable with objects of its subclasses without breaking the program. A subclass must honor the contract of the parent class.

Before (violates LSP)

Square inherits from Rectangle but changes the behavior in a surprising way.

```
public class Rectangle {
    protected int width;
    protected int height;

    public void setWidth(int width) { this.width = width; }
    public void setHeight(int height) { this.height = height; }
    public int getArea() { return width * height; }
}

public class Square extends Rectangle {
    @Override
    public void setWidth(int width) {
        this.width = width;
        this.height = width; // forces height = width
    }

    @Override
    public void setHeight(int height) {
        this.width = height;
        this.height = height;
    }
}

// Client code that expects a real Rectangle
public void testRectangle(Rectangle r) {
```

```
r.setWidth(5);
r.setHeight(4);

// Expects area = 20, but if r is a Square it becomes 16 → broken!

System.out.println(r.getArea());
}
```

After (follows LSP)

Don't force inheritance when the behaviors are different. Prefer composition or separate hierarchies.

```
public interface Shape {
    int getArea();
}

public class Rectangle implements Shape {
    private int width;
    private int height;

    public Rectangle(int width, int height) {
        this.width = width;
        this.height = height;
    }

    @Override
    public int getArea() {
        return width * height;
    }
}

public class Square implements Shape {
    private int side;
```

```
public Square(int side) {  
    this.side = side;  
}  
  
@Override  
public int getArea() {  
    return side * side;  
}  
}
```

Now any code that uses Shape works correctly with both Rectangle and Square.

4. Interface Segregation Principle (ISP)

Theory

Clients should not be forced to depend on methods they do not use. Prefer many small, specific interfaces over one large “fat” interface.

Before (violates ISP)

A big interface forces every class to implement methods it doesn't need.

```
public interface Worker {  
    void work();  
    void eat();  
    void sleep();  
}  
  
public class Robot implements Worker {  
    @Override  
    public void work() {  
        System.out.println("Robot working");  
    }  
  
    @Override  
    public void eat() {  
        // Robots don't eat → forced empty method  
    }  
}
```

```
}

@Override
public void sleep() {
    // Robots don't sleep → forced empty method
}
}
```

After (follows ISP)

Split into small interfaces.

```
public interface Workable {
    void work();
}

public interface Eatable {
    void eat();
}

public interface Sleepable {
    void sleep();
}

public class Human implements Workable, Eatable, Sleepable {
    public void work() { System.out.println("Human working"); }
    public void eat() { System.out.println("Human eating"); }
    public void sleep() { System.out.println("Human sleeping"); }
}

public class Robot implements Workable {
    public void work() { System.out.println("Robot working"); }
}
```

5. Dependency Inversion Principle (DIP)

Theory

High-level modules should not depend on low-level modules. Both should depend on abstractions.

Abstractions should not depend on details. Details should depend on abstractions.

Before (violates DIP)

The high-level class (NotificationService) depends directly on a concrete low-level class (EmailSender).

```
public class EmailSender {  
    public void send(String message) {  
        System.out.println("Sending email: " + message);  
    }  
}  
  
public class NotificationService {  
    private EmailSender emailSender = new EmailSender(); // tightly coupled  
  
    public void notify(String message) {  
        emailSender.send(message);  
    }  
}
```

If we later want SMS or Push notifications we must change NotificationService.

After (follows DIP)

Depend on an interface (abstraction). Concrete classes implement the interface.

```
public interface MessageSender {  
    void send(String message);  
}  
  
public class EmailSender implements MessageSender {  
    @Override  
    public void send(String message) {
```

```

        System.out.println("Sending email: " + message);
    }
}

public class SmsSender implements MessageSender {
    @Override
    public void send(String message) {
        System.out.println("Sending SMS: " + message);
    }
}

public class NotificationService {
    private final MessageSender sender; // depends on abstraction

    public NotificationService(MessageSender sender) {
        this.sender = sender; // injected from outside
    }

    public void notify(String message) {
        sender.send(message);
    }
}

// Usage
NotificationService emailService =
    new NotificationService(new EmailSender());
NotificationService smsService =
    new NotificationService(new SmsSender());

```

Quick Summary

Principle	Goal	Key idea in one sentence
-----------	------	--------------------------

S	One reason to change	One class → one job
O	Easy to extend	Add new code, don't change old code
L	Subtypes must behave correctly	Child class must not surprise the parent
I	Small interfaces	Don't force unused methods
D	Depend on abstractions	High-level code should not know low-level details